

STAR SPI Communication Protocol: Implementation Guide

nanopb, ARQ, and CRC-32 for ESP32-S3 Motor Control

Technical Documentation

December 16, 2025

Abstract

This document provides a comprehensive implementation guide for the STAR firmware SPI communication protocol. The protocol uses Protocol Buffers (nanopb) for serialization, Stop-and-Wait ARQ for reliability, and hardware-accelerated CRC-32 for error detection. All specifications are derived from the design document `Protobuf_Protocol_Design_Analysis.tex`.

The protocol achieves: (1) 99.9999% reliability with CRC-32, (2) $27\times$ throughput margin at 10 Mbps SPI with Stop-and-Wait ARQ, (3) <10 KB memory footprint with zero dynamic allocation via nanopb, and (4) <20 ms emergency stop latency for safety-critical motor control.

Contents

1 Protocol Stack Architecture

1.1 Layer Overview

The STAR protocol implements a four-layer stack over SPI peripheral mode:

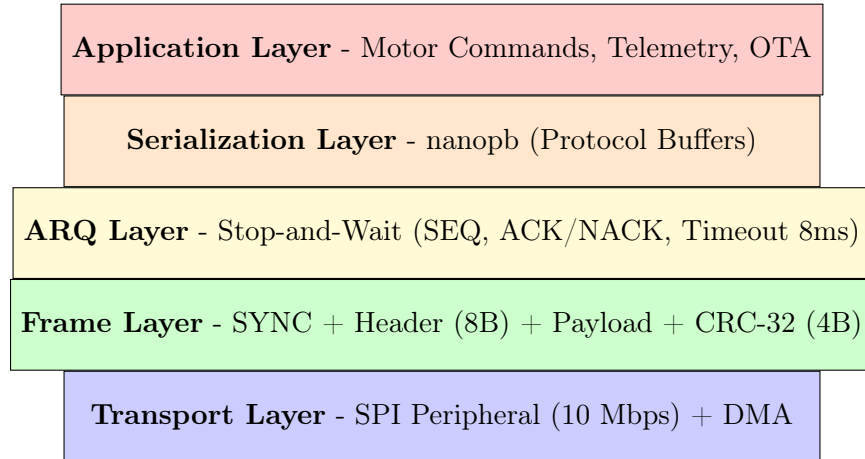


Figure 1: STAR Protocol Stack Layers

1.2 Design Requirements

Table 1: Protocol Performance Targets

Metric	Target	Rationale
Memory footprint	<10 KB	2% of 512 KB SRAM
Reliability	99.9999%	CRC-32 error detection
Throughput margin	27×	Stop-and-Wait sufficient
Emergency stop latency	<20 ms	Safety requirement
Dynamic allocation	Zero	Safety-critical system
SPI speed	10 Mbps	Custom PCB <15cm traces

2 Phase 1: CRC Library (lib/star_crc)

2.1 Hardware CRC-32 Acceleration

The ESP32-S3 provides ROM-based CRC-32 calculation using `esp_rom_crc32_le()` with IEEE 802.3 polynomial (0x04C11DB7). This is 5-10× faster than software CRC-16.

2.1.1 Directory Structure

```
lib/star_crc/  
  include/  
    star_crc.h  
  src/  
    star_crc.c  
  library.json  
  CMakeLists.txt
```

2.1.2 API Definition

```
1  #ifndef STAR_CRC_H  
2  #define STAR_CRC_H  
3  
4  #include <stdint.h>  
5  #include <stddef.h>  
6  #include "esp_err.h"  
7  
8  /**  
9   * @brief Initialize CRC-32 module  
10   * @return ESP_OK on success  
11   */  
12  esp_err_t star_crc_init(void);  
13  
14  /**  
15   * @brief Calculate CRC-32 checksum  
16   * @param data Pointer to data buffer  
17   * @param len Length of data in bytes  
18   * @return CRC-32 checksum (IEEE 802.3)  
19   */  
20  uint32_t star_crc_calculate(const uint8_t* data, size_t len);  
21  
22  /**  
23   * @brief Validate CRC-32 checksum  
24   * @param data Pointer to data buffer  
25   * @param len Length of data in bytes  
26   * @param expected_crc Expected CRC-32 value  
27   * @return true if CRC matches, false otherwise  
28   */  
29  bool star_crc_validate(const uint8_t* data, size_t len,  
30                        uint32_t expected_crc);  
31  
32  #endif // STAR_CRC_H
```

Listing 1: star_crc.h - CRC-32 Interface

2.1.3 Implementation

```

1 #include "star_crc.h"
2 #include "rom/crc.h" // ESP32-S3 ROM CRC functions
3 #include "esp_log.h"
4
5 static const char* TAG = "star_crc";
6
7 esp_err_t star_crc_init(void) {
8     ESP_LOGI(TAG, "CRC-32 module initialized (hardware-accelerated)")
9     ;
10    return ESP_OK;
11 }
12
13 uint32_t star_crc_calculate(const uint8_t* data, size_t len) {
14     if (data == NULL || len == 0) {
15         return 0;
16     }
17
18     // Use ESP32-S3 ROM CRC-32 (IEEE 802.3 polynomial)
19     return crc32_le(0, data, len);
20 }
21
22 bool star_crc_validate(const uint8_t* data, size_t len,
23                        uint32_t expected_crc) {
24     uint32_t calculated_crc = star_crc_calculate(data, len);
25     return (calculated_crc == expected_crc);
26 }

```

Listing 2: star_crc.c - CRC-32 Implementation

3 Phase 2: Protocol Buffers (lib/star_proto)

3.1 Directory Structure

```

lib/star_proto/
include/
    star_proto.h    # Main protocol interface
    star_frame.h    # Frame layer API
    star_arq.h      # ARQ layer API
src/
    star_frame.c    # Frame implementation
    star_arq.c      # ARQ implementation
generated/
    messages.pb.h   # nanopb-generated code
    messages.pb.c

```

```

proto/
  messages.proto      # Protobuf message schemas
  messages.options    # nanopb configuration
  library.json
  CMakeLists.txt

```

3.2 Message Schemas (messages.proto)

```

1 syntax = "proto3";
2
3 // ===== Commands (RPi5 -> ESP32) =====
4
5 message MotorSpeedCmd {
6     uint32 motor_id = 1;           // 0-3
7     float setpoint_rpm = 2;        // Target RPM
8 }
9
10 message MotorStopCmd {
11     uint32 motor_id = 1;           // 0-3
12     bool use_brake = 2;            // true=brake, false=coast
13 }
14
15 message EmergencyStopCmd {
16     // No fields - immediate stop all motors
17 }
18
19 message SetPidGainsCmd {
20     uint32 motor_id = 1;           // 0-3
21     float kp = 2;
22     float ki = 3;
23     float kd = 4;
24 }
25
26 message OtaWriteChunkCmd {
27     uint32 offset = 1;             // Byte offset in firmware
28     bytes data = 2;               // Chunk data (max 1024 bytes)
29 }
30
31 // ===== Responses (ESP32 -> RPi5) =====
32
33 message TelemetryResp {
34     repeated float motor_rpm = 1; // 4 motors
35     repeated float motor_current_ma = 2; // 4 motors
36     float temperature_c = 3;
37     float voltage_v = 4;
38 }
39

```

```

40 message StatusResp {
41     uint32 state = 1;           // System state
42     uint32 error_code = 2;      // Error code (0 = no error)
43 }
44
45 message AckResp {
46     uint32 seq_number = 1;      // Acknowledged sequence number
47 }
48
49 message NackResp {
50     uint32 seq_number = 1;      // Failed sequence number
51     uint32 error_code = 2;      // Reason for NACK
52 }

```

Listing 3: messages.proto - Protocol Buffer Definitions

3.3 Nanopb Options (messages.options)

```

1 # Maximum array sizes (static allocation)
2 TelemetryResp.motor_rpm max_count:4
3 TelemetryResp.motor_current_ma max_count:4
4
5 # Maximum OTA chunk size (1KB per design doc)
6 OtaWriteChunkCmd.data max_size:1024
7
8 # Enable fixed-length encoding
9 * int_size:IS_32

```

Listing 4: messages.options - Static Allocation Config

4 Phase 3: Frame Layer

4.1 Frame Structure

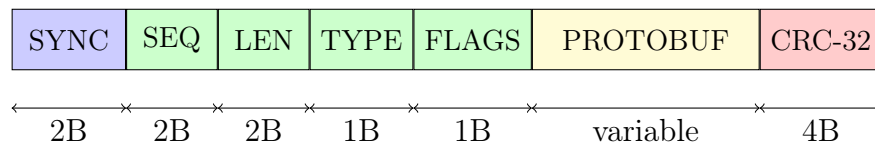


Figure 2: Frame Structure (Total: 12 + payload bytes)

Table 2: Frame Header Specification

Field	Size	Description
SYNC	2 bytes	0xAA55 (frame synchronization marker)
SEQ	2 bytes	Sequence number (0-65535, wraps)
LEN	2 bytes	Payload length (0-1024 for OTA chunks)
TYPE	1 byte	Message type (0x01=CMD, 0x02=RESP, 0x03=ACK, 0x04=NACK)
FLAGS	1 byte	Control flags (reserved for future use)
PAYLOAD	variable	nanopb-encoded protobuf message
CRC-32	4 bytes	IEEE 802.3 checksum (hardware-accelerated)

4.2 Frame Header Fields

4.3 Frame Layer API

```

1  #ifndef STAR_FRAME_H
2  #define STAR_FRAME_H
3
4  #include <stdint.h>
5  #include <stddef.h>
6  #include <stdbool.h>
7  #include "esp_err.h"
8
9  #define STAR_FRAME_SYNC      0xAA55
10 #define STAR_FRAME_HDR_SIZE  8      // SYNC(2) + SEQ(2) + LEN(2) +
    TYPE(1) + FLAGS(1)
11 #define STAR_FRAME_CRC_SIZE  4
12 #define STAR_FRAME_MAX_PAYLOAD 1040 // 1KB OTA chunk + margin
13
14 typedef enum {
15     STAR_FRAME_TYPE_CMD    = 0x01,
16     STAR_FRAME_TYPE_RESP   = 0x02,
17     STAR_FRAME_TYPE_ACK    = 0x03,
18     STAR_FRAME_TYPE_NACK   = 0x04,
19 } star_frame_type_t;
20
21 /**
22  * @brief Build frame from payload

```

```

23  * @param payload Protobuf-encoded payload
24  * @param payload_len Length of payload
25  * @param seq Sequence number
26  * @param type Frame type
27  * @param out_frame Output frame buffer
28  * @param out_len Output frame length
29  * @return ESP_OK on success
30  */
31  esp_err_t star_frame_build(const uint8_t* payload, size_t payload_len
32  ,
33  uint16_t seq, uint8_t type,
34  uint8_t* out_frame, size_t* out_len);
35  /**
36  * @brief Parse frame and extract payload
37  * @param frame Input frame buffer
38  * @param frame_len Length of frame
39  * @param out_payload Output payload buffer
40  * @param out_payload_len Output payload length
41  * @param out_seq Output sequence number
42  * @param out_type Output frame type
43  * @return ESP_OK on success
44  */
45  esp_err_t star_frame_parse(const uint8_t* frame, size_t frame_len,
46  uint8_t* out_payload, size_t*
47  out_payload_len,
48  uint16_t* out_seq, uint8_t* out_type);
49  /**
50  * @brief Validate frame CRC-32
51  * @param frame Frame buffer
52  * @param frame_len Frame length
53  * @return true if CRC valid, false otherwise
54  */
55  bool star_frame_validate(const uint8_t* frame, size_t frame_len);
56
57  #endif // STAR_FRAME_H

```

Listing 5: star_frame.h - Frame Layer Interface

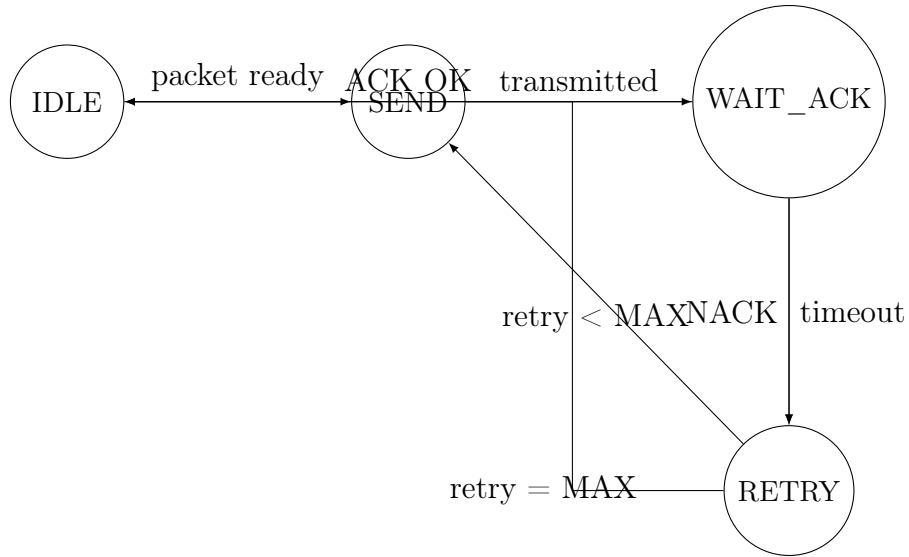


Figure 3: Stop-and-Wait Sender State Machine

Table 3: ARQ Parameters

Parameter	Value	Rationale
Timeout	8 ms	Design doc line 835
Max retries	3	Balance reliability vs latency
Sequence range	0-65535	16-bit wraparound
Window size	1	Stop-and-Wait (27× margin)

5 Phase 4: ARQ Layer

5.1 Stop-and-Wait State Machine

5.2 ARQ Configuration

5.3 ARQ Layer API

```

1 #ifndef STAR_ARQ_H
2 #define STAR_ARQ_H
3
4 #include <stdint.h>
5 #include <stddef.h>
6 #include "esp_err.h"
7 #include "star_frame.h"
8
9 #define STAR_ARQ_TIMEOUT_MS 8
10 #define STAR_ARQ_MAX_RETRIES 3
11
12 typedef struct star_arq_handle_s star_arq_handle_t;

```

```

13
14 typedef enum {
15     STAR_ARQ_STATE_IDLE,
16     STAR_ARQ_STATE_SEND,
17     STAR_ARQ_STATE_WAIT_ACK,
18     STAR_ARQ_STATE_RETRY,
19 } star_arq_state_t;
20
21 /**
22  * @brief Initialize ARQ layer
23  * @param handle ARQ handle
24  * @return ESP_OK on success
25  */
26 esp_err_t star_arq_init(star_arq_handle_t* handle);
27
28 /**
29  * @brief Send payload with ARQ
30  * @param handle ARQ handle
31  * @param payload Protobuf-encoded payload
32  * @param len Payload length
33  * @param type Frame type
34  * @return ESP_OK on success, ESP_FAIL on max retries
35  */
36 esp_err_t star_arq_send(star_arq_handle_t* handle,
37                         const uint8_t* payload, size_t len,
38                         uint8_t type);
39
40 /**
41  * @brief Receive payload with ARQ
42  * @param handle ARQ handle
43  * @param payload Output payload buffer
44  * @param len Output payload length
45  * @param type Output frame type
46  * @param timeout_ms Receive timeout
47  * @return ESP_OK on success
48  */
49 esp_err_t star_arq_receive(star_arq_handle_t* handle,
50                           uint8_t* payload, size_t* len,
51                           uint8_t* type, uint32_t timeout_ms);
52
53 /**
54  * @brief Process ACK/NACK
55  * @param handle ARQ handle
56  * @param seq Sequence number from ACK/NACK
57  * @param is_ack true for ACK, false for NACK
58  * @return ESP_OK on success
59  */

```

```

60 esp_err_t star_arq_process_ack(star_arq_handle_t* handle,
61                               uint16_t seq, bool is_ack);
62
63 /**
64  * @brief Get current sequence number
65  * @param handle ARQ handle
66  * @return Current sequence number
67  */
68 uint16_t star_arq_get_seq(star_arq_handle_t* handle);
69
70 #endif // STAR_ARQ_H

```

Listing 6: star_arq.h - ARQ Layer Interface

6 Phase 5: Communication Task

6.1 Architecture

6.2 Message Handlers

Table 4: Command Handlers

Message	Handler	Response
MotorSpeedCmd	queue_motor_command()	AckResp
MotorStopCmd	queue_motor_stop()	AckResp
EmergencyStopCmd	immediate_motor_stop()	AckResp
SetPidGainsCmd	update_pid_gains()	AckResp
OtaWriteChunkCmd	star_ota_write()	AckResp/NackResp
RequestTelemetry	encode_telemetry()	TelemetryResp

7 Phase 6: Build Integration

7.1 platformio.ini Updates

```

1 [env:esp32s3]
2 platform = espressif32
3 board = esp32-s3-devkitc-1
4 framework = espidf
5
6 lib_deps =
7     nanopb/Nanopb@~0.4.8
8
9 build_flags =

```

```

10     -DCONFIG_STAR_PROTO_ENABLE=1
11
12 extra_scripts =
13     pre:scripts/generate_proto.py

```

Listing 7: platformio.ini - Add nanopb Dependency

7.2 Proto Generation Script

```

1  #!/bin/bash
2  set -e
3
4  PROTO_DIR="lib/star_proto/proto"
5  OUT_DIR="lib/star_proto/src/generated"
6
7  # Find nanopb generator
8  NANOPB_GENERATOR=$(find ~/.platformio/packages -name "protoc-gen-
   nanopb" | head -n 1)
9
10 if [ -z "$NANOPB_GENERATOR" ]; then
11     echo "Error: nanopb generator not found"
12     exit 1
13 fi
14
15 # Generate C code from .proto
16 mkdir -p "$OUT_DIR"
17 protoc --plugin=protoc-gen-nanopb="$NANOPB_GENERATOR" \
18     --nanopb_out="$OUT_DIR" \
19     -I"$PROTO_DIR" \
20     "$PROTO_DIR/messages.proto"
21
22 echo "Proto generation complete: $OUT_DIR"

```

Listing 8: scripts/generate_proto.sh

8 Implementation Checklist

- ☐ Phase 1: Create lib/star_crc/ with hardware CRC-32
- ☐ Phase 2: Create lib/star_proto/ directory structure
- ☐ Phase 3: Write messages.proto and messages.options
- ☐ Phase 4: Implement star_frame.c (frame layer)
- ☐ Phase 5: Implement star_arq.c (ARQ layer)

- ☐ Phase 6: Rewrite `communication_task.c`
- ☐ Phase 7: Update `platformio.ini` with `nanopb`
- ☐ Phase 8: Create `scripts/generate_proto.sh`
- ☐ Phase 9: Integration testing with RPi5
- ☐ Phase 10: Performance validation (100 Hz, <20 ms emergency stop)

9 Success Criteria

Table 5: Validation Metrics

Metric	Target	Measurement Method
Memory usage	<10 KB	ESP-IDF heap/stack profiling
Reliability	99.9999%	CRC-32 error injection test
Throughput	100 Hz	Sustained motor command rate
Emergency stop	<20 ms	Logic analyzer timing
ARQ retry	>95% success	Error injection test

10 References

1. `Protobuf_Protocol_Design_Analysis.tex` - System design document
2. `nanopb` Documentation - <https://jpa.kapsi.fi/nanopb/>
3. ESP32-S3 Technical Reference Manual - Espressif Systems
4. Protocol Buffers Language Guide - Google

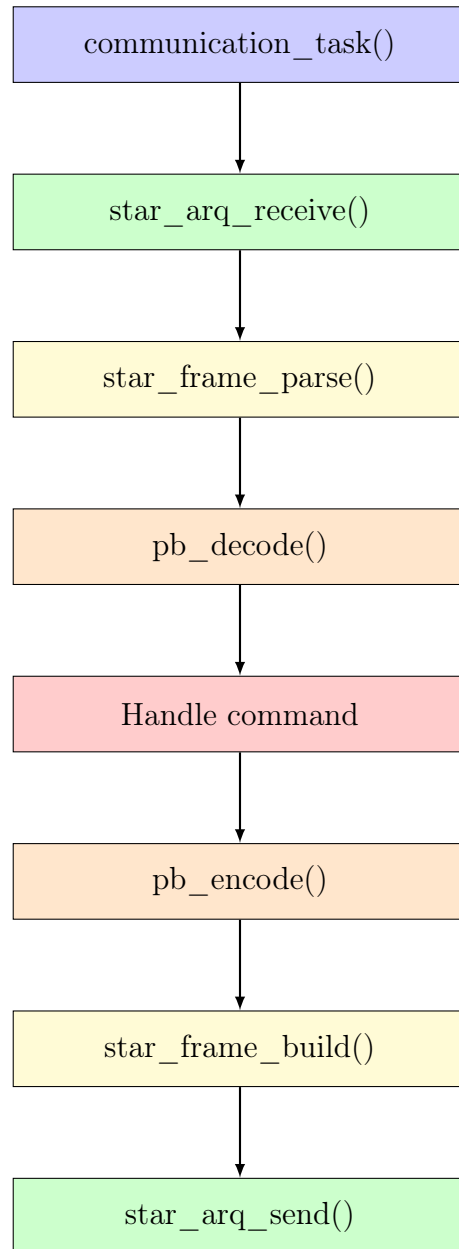


Figure 4: Communication Task Flow